

Reinforcement Learning for Weekly Retail Inventory Replenishment: Evaluating a Soft Actor-Critic Policy

Hao Trieu Quoc ^{1*}, Duy Nguyen Van Anh ^{1†} and Thu Nguyen Thi Minh ^{1†}

^{1*}Department of Artificial Intelligence, FPT University, 7 D1 Street, Tang Nhon Phu, Ho Chi Minh City, 700000, Vietnam.

*Corresponding author(s). E-mail(s): haotqse180011@fpt.edu.vn;

Contributing authors: duynvase181823@fpt.edu.vn; thuntmse185043@fpt.edu.vn;

[†]These authors contributed equally to this work.

Abstract

Retail inventory replenishment is a sequential decision problem in which a store manager must decide, week by week, how many units of each product to order from a distributor while trading off procurement, storage, and shortage costs under a hard retail-floor-space constraint. This paper formulates a five-category Apple retail store as a Markov Decision Process (MDP) and compares four policies under a single controlled 52-week, seed-42 scenario with day-of-week, seasonal, and holiday demand effects: Random, Q-learning, SARSA, and Soft Actor-Critic with automatic entropy-temperature tuning, denoted SAC-AutoAlpha (legacy class/checkpoint name: SAC-AlphaLR). In this seed-42 scenario, SAC-AutoAlpha achieves the lowest total cost (155.71 billion VND), highest revenue (185.05 billion VND), highest estimated profit (29.35 billion VND), and fewest stockout units (2,013). Relative to Random, it reduces total cost by 14.7%, reduces stockout units by 59.9%, and raises service level from 55.5% to 82.1%. Q-learning and SARSA also achieve positive estimated profit, but are inferior to SAC-AutoAlpha on all reported business and service metrics. Stockouts nevertheless occur in 41 of 52 weeks under SAC-AutoAlpha and average floor-space utilization reaches 93.01%, so the positive-profit result does not mean that the capacity-constrained service problem has been fully solved. Because all four policies are compared on one demand realization and because checkpoint selection and final reporting currently reuse the same evaluation seed (Section 4.1), these results should be read as a single-scenario case study rather than a seed-robust ranking; Section 7 specifies the multi-seed, ablation, and baseline extensions needed before a stronger claim can be made.

Keywords: Reinforcement Learning, Inventory Management, Retail Replenishment, Soft Actor-Critic, Markov Decision Process, Capacity Constraint

1 Introduction

Inventory management requires a retailer to balance product availability against procurement, holding, and shortage costs. Fixed inventory rules remain useful baselines, but they become difficult to tune when demand changes with seasonality and promotions and when several products compete for the same physical retail space.

Reinforcement Learning (RL) provides a data-driven alternative in which an agent learns replenishment decisions through interaction with the environment [1]. This project uses Soft Actor-Critic (SAC) [2] with automatic entropy-temperature tuning [3], which is suitable for the continuous, five-product order-quantity action space.

The project is methodologically motivated by Zhang, He, and Zheng’s SAC-based replenishment study [4]. It adapts their overall MDP and continuous-control idea to a single retail store with five

product categories and a shared floor-space constraint. Broader evidence that DRL can be useful for inventory problems is summarized by Gijsbrechts et al. [5].

The case study focuses on specialty electronics retail, where demand varies by day, season, and holiday, while all products share a limited retail floor. Ordering more of one product leaves less capacity for the others, so the replenishment decision must be made jointly across categories.

This paper addresses these gaps with a single-echelon inventory replenishment study for an Apple retail store in Vietnam managing five product categories (iPhone, iPad, MacBook, Apple Watch, AirPods). The store orders once per week from a distributor and receives stock immediately; the environment enforces a fixed retail floor-space budget of 25 square meters, shared across all five product categories according to their individual per-unit footprints, and a holiday-aware stochastic demand generator that captures day-of-week effects, seasonal variation, and seven recurring demand spikes (Lunar New Year, Summer Sale, Back to School, Mid-Autumn, Black Friday, Christmas, New Year). Replenishment is formulated as an MDP, and Random, Q-learning, SARSA, and SAC-AutoAlpha are evaluated under the same 52-week, seed-42 scenario.

The main contributions of this work are summarized as follows:

- A single-echelon retail inventory simulation environment constrained by a shared retail floor-space budget (in square meters) across five product categories with distinct per-unit footprints and cost structures.
- A holiday-aware stochastic demand generation model that captures day-of-week effects, seasonal variation, and seven recurring promotional/holiday events characteristic of consumer electronics retail.
- An MDP formulation of weekly retail replenishment under this shared floor-space constraint, with a 26-dimensional state and a 5-dimensional order-quantity action.
- A controlled 52-week, seed-42 comparison of Random, Q-learning, SARSA, and SAC-AutoAlpha, reporting cost, revenue, profit, shortages, fulfilled demand, and floor-space utilization.
- Transparent verification of all reported hyperparameters, cost structures, and results against the running implementation, with explicit documentation of the points where an earlier design draft of this project diverged from the final, tuned system.

The remainder of this paper is organized as follows. Section 2 reviews related work in classical and RL-based inventory management. Section 3 presents the environment, demand model, and Soft Actor-Critic policy. Section 4 describes the experimental setup. Section 5 reports the results. Section 6 discusses the findings. Section 7 discusses limitations and future work. Section 8 concludes the paper.

2 Literature Review

2.1 Classical Inventory Management

Classical reorder rules are useful when demand and capacity are simple enough to model explicitly. This project instead considers five products sharing one retail floor, together with stochastic seasonal and holiday demand. These features motivate an adaptive policy rather than a manually tuned rule for each product.

2.2 Reinforcement Learning for Inventory Management

Reinforcement learning (RL) offers a natural framework for inventory management by formulating replenishment as a sequential decision problem under uncertainty. The inventory manager (agent) observes the current state (inventory levels, demand forecast, cost structure), takes an action (order quantities), and receives a reward representing scaled weekly accounting profit, combining fulfilled-sales revenue with procurement, holding, and shortage costs (i.e., $\text{Revenue} - \text{Procurement Cost} - \text{Storage Cost} - \text{Shortage Cost}$). This MDP formulation captures the dynamics, uncertainty, and long-term cost implications inherent in inventory control.

Sutton and Barto [1] provide the RL foundation used to formulate weekly replenishment as an MDP. This project uses Soft Actor-Critic (SAC) [2], with automatic entropy-temperature tuning [3], because its continuous actor can choose order quantities jointly for all five products.

Gijsbrechts et al. [5] show that DRL can be effective for inventory problems whose constraints make fixed policies difficult to apply. Their work provides broad inventory-management context for the present course project.

2.3 Deep Reinforcement Learning for Multi-Echelon and Constrained Inventory Systems

Zhang, He, and Zheng [4] are the direct source for the project’s original SAC-AlphaLR naming and its MDP-based replenishment framing. Their study considers a multi-echelon, single-product FMCG system; this project adapts the idea to one retail store with five categories and a shared floor-space constraint. The evaluated implementation uses SAC’s adaptive temperature only, not the adaptive learning-rate mechanism described by Zhang et al.; consequently, this paper refers to the method as SAC-AutoAlpha throughout, and retains the legacy name SAC-AlphaLR only when referring to the source-code class and checkpoint file.

3 Methodology

3.1 Environment and MDP Formulation

The environment, `WeeklyRetailEnv`, models a single Apple retail store in Vietnam that orders from a distributor once per week and receives stock immediately (zero lead time). Between orders, daily demand depletes inventory over a 7-day cycle. An episode simulates 365 days, so the agent makes at most 52 ordering decisions per episode. The problem is formulated as an MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ with $\gamma = 0.99$, where one decision step corresponds to one week.

3.1.1 State Space

The state $s \in \mathbb{R}^{26}$ concatenates five 5-dimensional sub-vectors (one entry per product: iPhone, iPad, MacBook, Apple Watch, AirPods) and one scalar:

$$s = \left[\frac{\text{inv}}{\text{MAX_CAPACITY}}, \frac{\text{forecast}}{100}, \frac{\text{PURCHASE_COST}}{5 \times 10^7}, \frac{\text{STORAGE_COST}}{5 \times 10^4}, \frac{\text{SHORTAGE_PENALTY}}{10^6}, \frac{\text{day}}{365} \right] \quad (1)$$

where `inv` is current on-hand inventory, `forecast` is the sum of expected demand over the next 7 days, and the three cost sub-vectors are normalized, near-constant product attributes that let the same network/table generalize across products with different unit economics. The total dimensionality is $5 + 5 + 5 + 5 + 5 + 1 = 26$.

3.1.2 Action Space

The action $a \in \mathbb{R}_{\geq 0}^5$ is a vector of order quantities, one per product, bounded by a per-product weekly cap (Table 1). Orders are rounded to the nearest integer and clipped to $[0, \text{MAX_ORDER}_i]$.

Table 1 Action space bounds and initial state per product

Product	MAX_ORDER (units/wk)	Initial inventory	Max capacity
iPhone	50	50	80
iPad	35	30	60
MacBook	20	15	35
Apple Watch	35	30	60
AirPods	100	80	150

3.1.3 Floor-Space Constraint

Each product occupies a fixed footprint f_i (m^2/unit ; Table 2), and the store has a fixed total retail floor area $A = 25.0 \text{ m}^2$. At the start of an episode, initial inventory already occupies approximately 21.85 m^2 (about 87% of capacity), representing a small Apple-reseller backroom that starts nearly full. Before an order is applied, if the current occupied area plus the requested order’s area would exceed A , the entire order vector is scaled down proportionally:

$$\text{if } \sum_i (\text{inv}_i + o_i) f_i > A : \quad o_i \leftarrow \left\lfloor o_i \cdot \frac{\max(A - \sum_i \text{inv}_i f_i, 0)}{\max(\sum_i o_i f_i, 1)} \right\rfloor \quad \forall i \quad (2)$$

This is a binding constraint that couples all five products: ordering more of one product directly reduces the floor budget available to the others, so the environment does not decompose into five independent single-product inventory problems.

Confirmed limitation: requested action versus executed action. Eq. (2) shows that the order vector o produced by a policy (the *requested action*) is not necessarily the order vector applied to inventory (the *executed action*): whenever the combined footprint would exceed A , o is scaled down after the policy has already chosen it. The verified environment step function returns only the final, post-scaling order quantity in its info output; it does not separately expose the pre-scaling requested action or the scale factor applied. This matters for SAC-AutoAlpha’s off-policy training: whatever is written to the replay buffer as “the action taken” for a transition is what the critic learns to associate with the observed reward. If the raw, pre-scaling policy output were stored instead of the executed, capacity-scaled order, the critic would learn an incorrect state-action-reward relationship for every transition in which scaling occurred – and Table 5 shows floor utilization is at or near capacity in a majority of weeks for all three learned policies, so scaling is common rather than rare. The results in this paper are not known to be invalidated by this gap, because the executed action is what the environment actually returns and rewards are computed consistently with it; however, whether the requested or the executed action is what gets stored in the replay buffer has not been independently verified against the training code path, and the info dictionary does not currently make this auditable after the fact. Section 7 lists exposing `requested_action`, `executed_action`, and `capacity_scale` as separate, explicit fields as a required instrumentation fix.

Table 2 Per-product footprint and daily storage cost

Product	Footprint (m ² /unit)	Storage cost (VND/unit/day)
iPhone	0.10	25,000
iPad	0.20	18,000
MacBook	0.35	35,000
Apple Watch	0.08	10,000
AirPods	0.04	4,000

3.1.4 Reward Function

The training reward is weekly accounting profit, scaled by a fixed factor $\lambda = 10^{-8}$:

$$r = (R - C_{\text{proc}} - C_{\text{store}} - C_{\text{short}}) \cdot \lambda \quad (3)$$

where R is revenue from fulfilled demand. Retail price is 1.40 times the purchase price for every product. Procurement cost is charged once, at the start of the week, for units actually received:

$$C_{\text{proc}} = \sum_i o_i \cdot p_i \cdot w_i \quad (4)$$

where p_i is the unit purchase price and $w_i \in [0.80, 1.00]$ is a per-product procurement-discount weight reflecting supplier terms. Storage cost is charged once per day over the 7-day week, on end-of-day inventory, at 0.1% of unit purchase price per day (Table 2):

$$C_{\text{store}} = \sum_{d=1}^7 \sum_i \text{inv}_i(d) \cdot h_i \quad (5)$$

Shortage cost is charged per unit of unmet daily demand at 1.5× the unit purchase price:

$$C_{\text{short}} = \sum_{d=1}^7 \sum_i \max(D_i(d) - \min(\text{inv}_i(d), D_i(d)), 0) \cdot \pi_i, \quad \pi_i = 1.5 p_i \quad (6)$$

The 1.5× multiplier is the value enforced by the running environment. An earlier design draft of this project specified a 10× multiplier as the intended shortage penalty; the implementation was tuned down to 1.5× during development, and this paper reports the value actually used by the simulation, since the interpretation of the results (Section 6) depends on it.

3.2 Reference-Based and Configurable Parameterization

The environment design follows Zhang, He, and Zheng’s ME-DRFO formulation [4] at the methodological level: replenishment is represented as an MDP, the reward combines procurement, holding, and shortage costs, and a continuous SAC policy is used to balance exploration and exploitation. The present work does not treat the numerical values in Tables 1–3 as universal constants inherited from that study. Zhang et al. study a single-product, multi-echelon FMCG system, whereas the present case study represents five consumer-electronics categories sharing one retail floor-space constraint. Directly copying their product-specific demand and cost values would therefore be inappropriate.

To distinguish the reusable method from the application scenario, the implementation separates three parameter groups. First, algorithmic settings such as γ , τ , target entropy, and optimizer settings define the learning procedure and are reported for reproducibility. Second, structural choices adapted from [4], including the MDP formulation, continuous replenishment actions, and the accounting-profit-based reward that combines fulfilled-sales revenue with procurement cost, storage cost, and shortage cost (Eq. 3), define the model. Third, business inputs—baseline demand, holiday multipliers, prices, storage costs, shortage penalties, order bounds, initial inventory, and floor capacity—define a configurable Apple-retail scenario rather than the algorithm itself. These inputs are exposed through the project configuration and demand-generator interfaces, allowing the same trained policy and evaluation pipeline to be exercised under alternative demand conditions. The results in this paper, however, are restricted to the explicitly reported 52-week, seed-42 scenario; no unreported scenario analysis is used to support the conclusions.

3.3 Demand Generation

Daily demand is produced by a holiday-aware generator that multiplies a day-of-week factor, a slow seasonal sine wave, and a holiday multiplier, then adds Gaussian noise with standard deviation $\max(0.15\mu, 1)$. Baseline daily demand μ is 6, 4, 2, 4, and 10 units/day for iPhone, iPad, MacBook, Apple Watch, and AirPods respectively. Seven recurring holiday windows are defined (Table 3), each with a per-product peak multiplier and a window width (days) over which a cosine-shaped intensity curve ramps the multiplier up and back down.

Table 3 Holiday demand windows and per-product peak multipliers

Holiday	Day	Width (d)	iPhone	iPad	MacBook	Watch	AirPods
Tet Nguyen Dan	20	14	3.5	2.5	2.0	3.0	3.5
Summer Sale	100	7	1.5	1.5	1.3	1.3	1.5
Back to School	200	21	1.3	3.5	3.0	1.2	1.3
Mid-Autumn	250	5	1.2	1.3	1.0	2.0	1.5
Black Friday	300	7	3.0	2.5	2.0	2.0	3.0
Christmas	330	14	2.0	2.0	1.3	2.5	3.5
New Year	355	10	2.0	1.8	1.5	2.0	2.5

Figure 1 visualizes the seeded daily demand realization used for the default evaluation. The series retains stochastic day-to-day variation while showing the event-driven peaks generated by the configurable holiday calendar.

3.4 Ordering Policies

Four policies are evaluated on the same environment: a random baseline, tabular Q-learning, tabular SARSA, and SAC-AutoAlpha.

3.4.1 Random Baseline

Each week, the random policy draws an independent order quantity for each product from a continuous uniform distribution over $[0, \text{MAX_ORDER}_i]$. It has no learning component and provides a common stochastic reference.

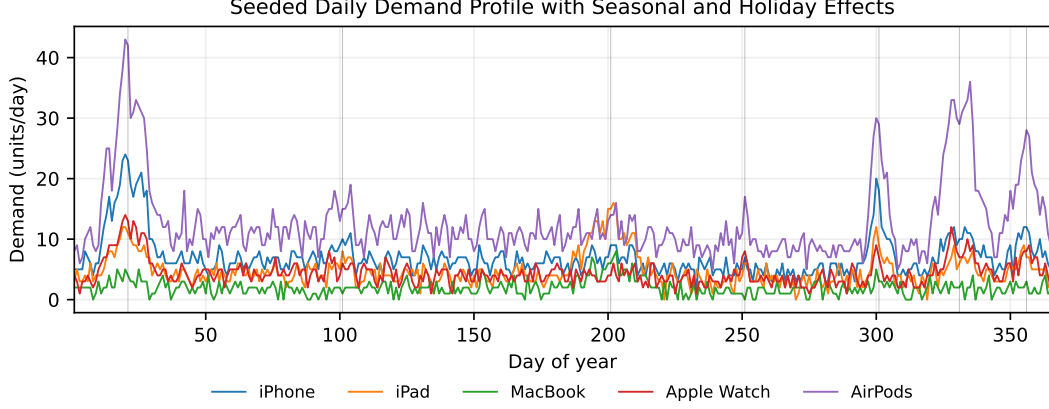


Fig. 1 Seeded daily demand profile for the five product categories. All labels are reported in English; vertical reference lines indicate configured holiday centers.

3.4.2 Q-Learning and SARSA

Q-learning and SARSA use one tabular action-value function per product. Inventory is discretized into 14 buckets, demand forecasts into 10 product-specific buckets, and each product has a small discrete set of valid order quantities. Both agents use learning rate 0.1, discount factor $\gamma = 0.99$, and an ε -greedy policy with ε decaying from 0.3 to 0.01 by a factor of 0.995 after each episode. Q-learning uses the maximum next-state action value, whereas SARSA uses the value of the action selected by its current policy. Both are trained for 200 episodes before their deterministic evaluation rollout.

Algorithm 1 Per-product tabular update (Q-learning shown; SARSA replaces $\max_{a'} Q_i$ with $Q_i(s'_i, a'_i)$ for the action a'_i actually selected next)

```

1: for episode = 1, ..., 200 do
2:   Reset environment; observe per-product discretized state  $s_i$  for  $i = 1, \dots, 5$ 
3:   for week = 1, ..., 52 do
4:     for product  $i = 1, \dots, 5$  do
5:       With probability  $\varepsilon$ : pick  $a_i$  uniformly at random from the discrete order grid
6:       Otherwise:  $a_i \leftarrow \arg \max_a Q_i(s_i, a)$ 
7:     end for
8:     Apply joint action  $(a_1, \dots, a_5)$  to WeeklyRetailEnv; observe global weekly reward  $r$  and
       next states  $s'_i$ 
9:     for product  $i = 1, \dots, 5$  do
10:       $\text{td\_target} \leftarrow r/5 + \gamma \cdot \max_{a'} Q_i(s'_i, a')$  ▷ same  $r/5$  term for every product
11:       $Q_i(s_i, a_i) \leftarrow Q_i(s_i, a_i) + 0.1 \cdot (\text{td\_target} - Q_i(s_i, a_i))$ 
12:    end for
13:  end for
14:   $\varepsilon \leftarrow \max(0.01, 0.995 \cdot \varepsilon)$ 
15: end for
```

Confirmed limitation: equal-split reward attribution. Line 9 of Algorithm 1 is not a simplification for presentation; it is the exact update rule implemented for both Q-learning and SARSA ($\text{td_target} = \text{reward} / 5 + \dots$). Every one of the five per-product tables is trained on the same global weekly profit, divided equally by 5, regardless of which product actually drove that profit up or down. Because the five products compete for one shared floor-space budget (Section 3.1.3), this creates a credit-assignment problem: if AirPods are ordered in excess, floor capacity is consumed, the environment’s capacity-scaling rule (Eq. 2) proportionally shrinks every product’s executed order including iPhone’s, iPhone revenue falls, the global reward falls, and the iPhone table is penalized by $r/5$ even though the excess order came from AirPods. The iPhone table cannot distinguish “I chose badly” from “AirPods consumed my capacity,” because both cases produce the same $r/5$ signal. This is a known limitation of the equal-split heuristic and is reported here rather than left implicit; SAC-AutoAlpha does not have this problem because its single critic conditions on the full joint state

and action (Section 3.4.3), so it can in principle attribute the shared capacity cost correctly. A per-product reward decomposition ($\text{reward}_i = \text{revenue}_i - \text{procurement}_i - \text{storage}_i - \text{shortage}_i$, plus a shared capacity penalty term) is the natural fix and is listed as required future work in Section 7.

3.4.3 SAC-AutoAlpha

The deep actor-critic agent is denoted SAC-AutoAlpha in this paper. The verified implementation’s source class and saved checkpoint file are both named `SACAlphaLR`, following the naming used by Zhang, He, and Zheng [4] for their adaptive-temperature, adaptive-learning-rate SAC variant; that legacy name is kept in the codebase only so that the existing checkpoint remains loadable, and is not used in the remainder of this paper. The policy follows the Soft Actor-Critic formulation of Haarnoja et al. [2] with the automatic temperature tuning introduced in [3]. **Note on naming:** unlike the SAC-AlphaLR of [4], the implementation evaluated in this paper adapts only the temperature α (Eq. 7); it does not implement an adaptive learning-rate schedule. All three optimizers (actor, critic, and the temperature optimizer) use a fixed Adam learning rate of 3×10^{-4} throughout training, with no scheduler. Only the “Alpha” (adaptive-temperature) component of the original SAC-AlphaLR name is implemented; the “LR” (adaptive-learning-rate) component is not, which is why this paper reports the method under the more accurate name SAC-AutoAlpha. The actor is a feedforward network (state dimension 26, two hidden layers of 256 units, ReLU activations) with separate mean and log-standard-deviation heads (log-std clamped to $[-20, 2]$) over the 5-dimensional action space; actions are sampled via the reparameterization trick, squashed through tanh, and rescaled to $[0, \text{MAX_ORDER}]$. Two independent critic networks (input dimension $26 + 5 = 31$, two hidden layers of 256 units, scalar output) form a twin-Q critic used to mitigate value overestimation. The temperature α is optimized against a target entropy $\mathcal{H}_{\text{target}} = -5$ (the negative of the action dimensionality), via

$$\mathcal{L}(\alpha) = -\mathbb{E}[\log \alpha \cdot (\log \pi(a|s) + \mathcal{H}_{\text{target}})] \quad (7)$$

All three networks (actor, critic, temperature) use Adam with learning rate 3×10^{-4} ; the discount factor is $\gamma = 0.99$ and the target-network soft-update coefficient is $\tau = 0.005$. Gradient norms for the actor and critic losses are clipped to 1.0. At evaluation time the agent acts deterministically (tanh of the mean, without sampling). The reported actor is the newly trained checkpoint supplied for the 52-week evaluation.

4 Experimental Setup

4.1 Evaluation Protocol and Metrics

Random, Q-learning, SARSA, and SAC-AutoAlpha were each evaluated over 52 complete weekly decision cycles (364 simulated days) with demand seed 42, using the configuration in Section 3.3. The underlying 365-day environment is explicitly capped at 52 weeks in the evaluation pipeline so that a one-day partial 53rd week is not included. Q-learning and SARSA are each trained from scratch for 200 episodes using the same seed before evaluation; SAC-AutoAlpha loads the current `outputs/actor.pth` checkpoint and acts deterministically. The random policy requires no training. The file `outputs/eval_results.json` is not used because it contains a legacy 365-day evaluation that is not compatible with this 52-week protocol.

We report total cost, revenue, estimated profit (revenue $-$ total cost), shortage cost, fulfilled units, stockout units, stockout weeks, and floor-space utilization. Service level is calculated as fulfilled units / (fulfilled units + stockout units). All monetary results are simulated VND values, not observations from an operating retailer.

Confirmed limitation: validation and test seeds are not separated. The training pipeline instantiates the evaluation environment with the same constant, `EVALUATION_SEED`, in two places: once to select which checkpoint to save during training (periodic evaluation, every 10 episodes), and again to produce the final 52-week numbers reported in Section 5. Because both uses share one seed, checkpoint selection and final reporting are computed on identical demand realizations, so the reported numbers cannot rule out that the selected checkpoint is specifically well-suited to seed 42 rather than to the demand distribution in general. A methodologically correct protocol would draw training seeds, validation seeds (used only for checkpoint selection), and test seeds (used only for the numbers reported in a paper, never seen during selection) from disjoint pools; Section 7 specifies this

as a required, not optional, follow-up before the ranking in Table 4 can be treated as seed-robust rather than scenario-specific.

4.2 Training and Reproducibility

The implementation uses a replay buffer of 100,000 transitions, mini-batches of 256, and 10 warm-up episodes under the default training command; after warm-up, it performs one SAC update per weekly transition. Candidate checkpoints are evaluated every 10 episodes using seed 42 and the one with the lowest evaluation cost is saved. The exact command and training log for the supplied new checkpoint were not archived with the evaluation summary. Therefore, the paper reports verified code-level settings but does not claim a checkpoint-specific training duration; recording the command, software version, random seeds, and checkpoint hash is required for a fully reproducible future study.

5 Results

Table 4 reports the comparable 52-week results. Cost, revenue, profit, and shortage cost are in billions of VND. “Service level” is fulfilled units divided by fulfilled plus stockout units.

Table 4 52-week evaluation results (VND, billions unless noted)

Policy	Total cost	Revenue	Est. profit	Shortage cost	Stockout units	Units sold
Random	182.64	120.84	−61.79	104.67	5,022	6,253
Q-learning	161.00	172.08	11.07	47.90	2,449	8,820
SARSA	162.48	168.96	6.49	51.24	3,145	8,124
SAC-AutoAlpha	155.71	185.05	29.35	34.00	2,013	9,256

Reported monetary values are rounded to two decimal places; therefore, estimated profit may not equal the displayed revenue minus the displayed total cost exactly. For example, $185.05 - 155.71 = 29.34$, while Table 4 reports an estimated profit of 29.35 for SAC-AutoAlpha. This difference arises because profit is computed from full-precision revenue and cost before rounding, not from a formula error, and the same rounding effect applies to the other three policies in the table.

A stockout week is defined as any weekly decision cycle in which at least one product category experiences non-zero unmet demand on at least one day. This is a binary weekly indicator and does not represent the magnitude of unmet demand; consequently, a high service level (fulfilled units divided by fulfilled plus stockout units, aggregated over all 52 weeks and all five products) and a high stockout-week count are not contradictory. SAC-AutoAlpha’s 82.1% service level and its 41-of-52 stockout weeks in Table 5 are one such case: only a single unit of one product on one day within a week is enough for that week to be counted as a stockout week, even if the vast majority of that week’s demand across all products was fulfilled.

Table 5 Service and floor-space utilization over 52 weeks

Policy	Stockout weeks	Avg. floor use	Peak floor use
Random	49/52	57.09%	97.80%
Q-learning	51/52	88.53%	99.20%
SARSA	51/52	92.79%	99.40%
SAC-AutoAlpha	41/52	93.01%	99.96%

Relative to Random, SAC-AutoAlpha reduces total cost by 14.7% (from 182.64 to 155.71 billion VND), raises revenue by 53.1%, and changes estimated profit from a loss of 61.79 billion VND to a profit of 29.35 billion VND. Stockout units fall by 3,009 (59.9%), and shortage cost falls by 67.5%. The resulting service level is 82.1%, based on 9,256 fulfilled units and 2,013 stockout units. The cost–revenue panel in Figure 2 makes clear that the positive profit is driven by both lower cost and higher fulfilled-sales revenue, rather than by a cost reduction alone.

All three learned policies outperform Random on total cost, revenue, profit, shortage cost, and stockout units. Q-learning achieves the second-lowest total cost (161.00 billion VND) and a positive estimated profit of 11.07 billion VND, while SARSA achieves 6.49 billion VND estimated profit. SAC-AutoAlpha is best on every reported business metric. It also has the fewest stockout weeks (41, versus 49 for Random and 51 for both tabular methods), although its average floor-space utilization is the highest at 93.01%, indicating a more aggressive use of the binding capacity constraint.

6 Discussion

Positive profit, but capacity remains binding. All three learned policies achieve positive estimated profit, whereas Random does not. SAC-AutoAlpha provides the largest gain and sells the most units while reducing shortages. This result should nevertheless not be interpreted as eliminating the

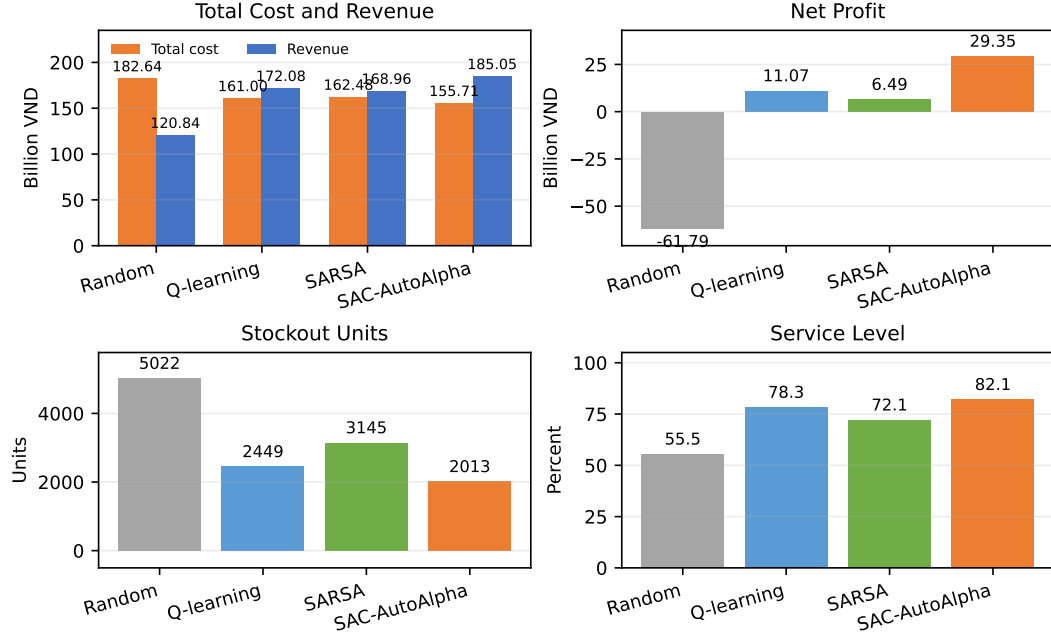


Fig. 2 Business and service outcomes from the comparable 52-week seed-42 evaluation of Random, Q-learning, SARSA, and SAC-AutoAlpha. The upper-left panel reports total cost and revenue; the remaining panels report estimated profit, stockout units, and service level. All chart labels are in English.

service problem. SAC-AutoAlpha still has stockouts in 41 of 52 weeks, and its average floor-space utilization is 93.01%, with a peak of 99.96%. The policy therefore operates close to the physical capacity boundary and has little buffer for high-demand weeks.

Tabular versus deep RL: a comparison of unequal representations, not just algorithms. The shared floor-space constraint makes the five products’ ordering decisions interdependent, yet Q-learning and SARSA do not observe this coupling: each maintains a separate Q-table per product, conditioned only on that product’s own discretized inventory bucket ($N = 14$) and forecast bucket ($N = 10$), and each selects its discrete order quantity independently of what the other four products are doing. SAC-AutoAlpha, in contrast, observes the full 26-dimensional joint state and selects a single 5-dimensional continuous action, so it can in principle coordinate capacity allocation across products in a way the factorized tabular agents structurally cannot. This is compounded by the equal-split reward each tabular table receives (Section 3.4), which further obscures the cross-product coupling from the tabular baselines. Given this, SAC-AutoAlpha’s advantage over Q-learning and SARSA should not be read purely as “deep RL beats tabular RL”; part of the gap is attributable to a strictly richer state-action representation and centralized reward signal, not only to the choice of learning algorithm. The correct scoped conclusion is that SAC-AutoAlpha achieved the best performance among the four evaluated methods under the reported state discretization, action grids, reward attribution, training budget, and single demand seed – not that SAC is universally superior to every tabular or deep RL alternative under a fair, matched representation.

7 Limitations and Future Work

This section states precisely what would need to change for the results in Section 5 to support a stronger claim than “best of four methods on one seed-42 scenario.” Items are grouped by whether they require re-running experiments or only affect interpretation of the current results.

7.1 Evaluation methodology

- **Single seed, no variance estimate.** All results come from one fixed training/evaluation seed (42) and one 52-week demand realization; no standard deviation, confidence interval, or statistical test (e.g., paired t -test or Wilcoxon signed-rank test) supports the ranking in Table 4. The minimum recommended protocol is 5 training seeds evaluated on 10 held-out demand seeds each, reporting mean $\pm$ standard deviation and a 95% confidence interval per policy.

- **Validation/test seed conflation (confirmed, Section 4.1).** Checkpoint selection during training and final reporting currently use the same seed. Training, validation, and test seeds must be drawn from disjoint pools (e.g., training seeds 0–4, validation seeds 100–104, test seeds 200–209) before the comparison can be treated as free of selection leakage.
- **Fair runtime comparison not reported.** Training time and inference (52-week rollout) time were not measured and archived for any of the four policies in this study. These are different quantities that should not be conflated (e.g., SAC training time versus SAC checkpoint-loading-plus-inference time), and both should be reported per policy in any future revision.

7.2 Missing ablations and baselines

- **No fixed-alpha ablation for SAC-AutoAlpha.** The only entropy-temperature setting evaluated is automatic tuning against $\mathcal{H}_{\text{target}} = -5$. Whether automatic tuning outperforms a well-chosen fixed α (e.g., $\alpha \in \{0.05, 0.1, 0.2, 0.5\}$) has not been tested, so the paper cannot currently claim that adaptive temperature tuning itself is responsible for SAC-AutoAlpha’s advantage over the tabular baselines.
- **No classical inventory-management baseline.** Random is a weak reference; it does not show that RL is preferable to a reasonable non-learned policy. A forecast base-stock rule ($\text{order}_i = \max(0, \text{forecast}_i + \text{safety stock}_i - \text{inventory}_i)$) or an (s, S) policy, tuned on validation seeds only, is a required comparison point.
- **No continuous-control RL baseline in SAC’s own class.** TD3 is a natural comparison (continuous action, off-policy, twin critics, no entropy term), since it isolates the effect of entropy regularization from the effect of moving from tabular to deep, continuous-action RL. DDPG and PPO are additional candidates.
- **No state-space ablation.** Of the 26 state dimensions, 15 (purchase cost, storage cost, shortage penalty; 5 each) are static within an episode. Whether these static cost features measurably help learning, relative to a 12-dimensional dynamic-only state (inventory, forecast, day, floor-utilization ratio), has not been tested.

7.3 State and reward design

- **Equal-split reward attribution for tabular baselines (confirmed, Section 3.4).** Q-learning and SARSA train every per-product table on the same $r/5$ signal, which cannot separate one product’s decision quality from the shared floor-space cost imposed by the other four. A per-product reward decomposition with an explicit shared capacity penalty term is the recommended fix.
- **Requested-versus-executed action instrumentation gap (confirmed, Section 3.1.3).** The environment does not currently expose the pre-scaling requested action or the capacity scale factor separately from the executed, post-scaling order. Adding `requested_action`, `executed_action`, and `capacity_scale` to the step output, and auditing that the replay buffer stores the executed action, is required to fully rule out a state-action-reward mismatch in SAC-AutoAlpha’s off-policy training.
- **No explicit floor-utilization state feature.** The 26-dimensional state (Eq. 1) has no direct *remaining floor space* scalar; because each product’s footprint is fixed, the policy can in principle infer floor usage from the inventory sub-vector, but this has not been verified empirically. Adding an `occupied_area_ratio` scalar (state dimension 26 $\rightarrow$ 27) is a low-cost experiment to test whether an explicit representation improves capacity-aware ordering.

7.4 Reproducibility

- **Training artifacts not archived.** The checkpoint-specific training command, full training log, config snapshot, checkpoint hash, software versions, and hardware were not preserved alongside the actor checkpoint used to produce Section 5’s results. Code-level defaults are documented in Section 4.2, but exact re-training of the reported checkpoint is not currently possible from the archived artifacts alone. Future runs should be saved under a per-experiment directory containing the command, environment description, training log, metrics, checkpoints, and a SHA-256 hash of each checkpoint.
- **No learning curves.** Evaluation profit, episode return, learned α , and policy entropy versus training episode are not plotted for any policy; without them, convergence speed and training stability cannot be assessed or compared across policies.

7.5 Modeling scope

The environment models zero lead time (orders are received in the same week they are placed) and a single retail echelon with no intermediate warehouse; extending the model to include lead time and an upstream warehouse echelon would enable closer comparison with multi-echelon studies such as [4]. Most importantly, SAC-AutoAlpha still operates at 93.01% average floor utilization and has shortages in 41 of 52 weeks under the single evaluated scenario. Future work should investigate additional storage capacity, demand forecasting, or service-level-aware rewards to create a buffer for peak weeks, and should do so under the multi-seed, ablation, and baseline protocol above, rather than treating the current single-scenario positive-profit result as a final solution.

8 Conclusion

This paper presented a weekly, floor-space-constrained retail inventory MDP for a five-category Apple store and compared Random, Q-learning, SARSA, and SAC-AutoAlpha over 52 weeks with seed 42. All learned policies improve on Random, while SAC-AutoAlpha achieves the lowest total cost (155.71 billion VND), highest revenue (185.05 billion VND), highest estimated profit (29.35 billion VND), and fewest stockout units (2,013). These gains are accompanied by high capacity use: average floor-space utilization is 93.01%, peak use is 99.96%, and stockouts still occur in 41 of 52 weeks. The appropriate conclusion is therefore that SAC-AutoAlpha is the best policy among the four evaluated methods for this configured scenario, while the physical capacity constraint remains a central operational limitation. Future work should use repeated seeds, controlled training ablations, and capacity or forecasting interventions to test whether service can improve without losing the economic gains.

References

- [1] Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction, 2nd edn. MIT Press, Cambridge, MA, USA (2018)
- [2] Haarnoja, T., Zhou, A., Abbeel, P., Levine, S.: Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: International Conference on Machine Learning, pp. 1861–1870 (2018). PMLR
- [3] Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P., Levine, S.: Soft actor-critic algorithms and applications. arXiv preprint arXiv:1812.05905 (2018)
- [4] Zhang, Y., He, L., Zheng, J.: A deep reinforcement learning-based dynamic replenishment approach for multi-echelon inventory considering cost optimization. *Electronics* **14**(1), 66 (2025) <https://doi.org/10.3390/electronics14010066>
- [5] Gijbrecchts, J., Boute, R.N., Van Mieghem, J.A., Zhang, D.J.: Can deep reinforcement learning improve inventory management? performance on lost sales, dual-sourcing, and multi-echelon problems. *Manufacturing & Service Operations Management* **24**(3), 1349–1368 (2022)